# Content and User Preference Based Music Recommendation System Using Knowledge Graphs and GNN

Group 7: Pedro Fanica (54346) Aarthi Perumpillichira (66404) Maja Skwaroń (66083)

## Abstract

*We present a music recommendation system that integrates symbolic music features, knowledge graph semantics, and graph neural networks. Starting from the Lakh MIDI Dataset and the Million Song Dataset, we construct a two-tier RDF knowledge graph—simple and rich variants—encoding 7,013 tracks, 3,668 artists, 1,046 genres, and 285,037 users, with ontological classes aligned to Wikidata via OWL semantics. Symbolic features (1,525 jSymbolic/music21 descriptors per track) are compressed to 128 dimensions via a fully connected autoencoder; complementary KG embeddings (RotatE [1], ComplEx [2]) initialise node features for all entity types. The graph is consumed by a Heterogeneous Graph Transformer (HGT) [3] trained with a custom intensity-weighted listwise loss incorporating logit adjustment for popularity debiasing. The knowledge graph's structural integrity is verified through GraphDB and SPARQL inspection. Evaluation against popularity-based, KNN collaborative filtering, and XGBoost hybrid baselines shows that KNN-CF achieves Recall@10 of 0.104 and NDCG@10 of 0.106, substantially outperforming popularity-based recommendation (NDCG@10 = 0.098, Coverage = 0.2%). Our framework demonstrates how structured musical knowledge and relational graph semantics can complement behavioural signals for more diverse, content-aware music recommendation.*

## 1. Introduction

**This project investigates how a Graph Neural Network (GNN)-based music knowledge graph, combining musical features (e.g., pitch statistics and tempo) as node embeddings, song metadata (such as artists and genres) as graph structure, and user interaction history as supervision, can be used to predict user-specific music preferences.**

Although many music recommendation algorithms already exist and are widely used, most of them rely primarily on user interaction data and similarity measures between users. While suggestions based on mutual listening history overlap can be quite effective, this approach remains inherently limited and strongly favors music and artists that are already popular. Musical preference is highly subjective, and relying solely on behavioral patterns may overlook important aspects of the music itself.

In reality, musical pieces are a rich source of information, differing from one another across multiple dimensions. Each piece can be characterized by a wide range of features, from more direct attributes such as the artist or genre to strictly musical properties derived through analysis, such as pitch statistics, tempo, or chord progressions. Incorporating these musical features into the recommendation process may provide a richer and more structured representation of songs, enabling models to capture deeper relationships between musical content and user preferences.

Because music is both an art form and a structured domain, it is particularly well-suited for computational analysis and representation as a knowledge graph. By leveraging a knowledge graph in combination with Graph Neural Networks (GNNs), this project aims to model complex, multi-relational dependencies and explore their potential for innovative, more accurate and context-aware music preference prediction.

## 2. Data

### 2.1. Data Sources

This project integrates data from the two main sources:

- **Lakh MIDI Dataset (LMD)**[4] - A large-scale collection of MIDI files (machine-readable representation of musical structure) representing music in symbolic form (notes, timing, instruments.
- **Million Song Dataset (MSD)**[5][6] - A dataset developed by The Echo Nest, containing audio-derived features such as tempo, key, or loudness metadata for songs, as well as a user taste profile subset.

### 2.2. Data Processing

The first step of the data processing methodology involved the extraction and linking of the Lakh MIDI and Million Song Dataset (MSD). The alignment between MIDI files

and audio tracks was based on precomputed similarity scores obtained using Dynamic Time Warping (DTW) by Raffel (2016). In this process, both MIDI and audio signals were first converted into chroma-based representations over time, and then compared to determine their similarity. The threshold for acceptable similarity score of 0.55 was set during the filtering step, in order to ensure reliable alignment. The remaining files and duplicates were omitted resulting in a dataset of 19,037 tracks, with each one described at this stage by 40 metadata fields which included track-level identifiers (e.g. track_id, song_id, or artist_id), descriptive fields (e.g. title, artist name, album, release year), tonal estimates (key, mode, and their confidences), rhythmic/audio scalars (time signature, loudness), genre/tag information (e.g. primary genre, top-3 genres), and MIDI-derived instrumentation (number and names of instruments). The exploratory overview of the MSD features is available in Figure 7 of the Appendix.

The next step concerned processing and subsequently linking user interaction data extracted from Echo Nest Taste Profile, represented in the form of (`user_id`, `song_id`, `play_count`) triplets. The raw dataset consisted of approximately 48 million interactions across over one million users and almost four hundred thousand songs. Interactions associated with known erroneous song ID mappings were removed using the official MSD mismatch list, which identified 18,913 incorrectly linked song IDs, of which 6,237 were present in the taste profile, accounting for 5.33% of all interactions. Subsequently, the user data was filtered to retain only the songs that also exist in the Lakh/MSD previously matched dataset, which constituted approximately 40.4% of all LMD songs. Additionally, instances with fewer than 5 remaining song interactions were removed to address the cold start problem, excluding users with insufficient listening history to train and evaluate recommendations for them. After the filtering step the dataset consisted of 299,156 unique users and 7,227 unique songs. Figure 1 illustrates the filtering funnel across three dimensions: total interactions, unique users, and unique songs, at each step of the filtering pipeline.

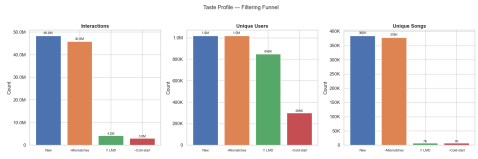


Figure 1. Taste Profile filtering funnel.

The detailed filtering statistics across all steps, together with filtered dataset diagnostics are summarized in Table 1 and 8 of the Appendix.

After the data integration and filtering process, feature extraction was performed using the jSymbolic [7] and music21 [8] extractors, which work directly with symbolic MIDI representations and produce high-level descriptors grounded in music theory. Out of the 19,037 matched tracks, 7,013 were successfully processed, forming the basis for all downstream modelling.

For each track jSymbolic and music21 together produced 1,525 numerical descriptors spanning seven categories: **pitch** (histograms, range, most-common-pitch statistics), **melodic** (interval histograms, arc direction and length), **harmonic** (simultaneous pitch distances), **rhythmic** (beat histograms, tempo statistics, note density), **dynamics** (loudness range, note-to-note variation), **textural** (voice count, voice equality), and **instrumentation**. The high dimensionality arises primarily from multi-bin histogram descriptors; the remaining features were approximately 150 scalars per track.

## 3. Methodology

### 3.1. Knowledge Graph Design and Construction

One of the main challenges and core components of this project was designing and populating the knowledge graph that would be used to train the recommendation algorithm. To support this objective, the knowledge graph had to be designed carefully to accommodate both diverse musical attributes and user behavior, as well as the relational structure necessary for effective representation learning.

The knowledge graph was built on top of an existing music ontology [9], with classes and properties tailored to music recommendation systems. Standard vocabularies expressed in RDF such as FOAF for agents, or event ontology for listening events were reused to ensure that the graph is interoperable with other Semantic Web tools and that its structure is transparent to readers familiar with these standards. A small number of project-specific relations were added under a dedicated *mrc:* namespace to cover concepts not present in the existing vocabularies, such as the link between a *performance* and its *tempo* category, or between a *listening* event and its play count.

#### 3.1.1. Simple Knowledge Graph

The knowledge graph was built from accurately matched tracks described by both MSD metadata and successfully extracted musical features. Not all metadata fields available in the merged dataset were retained for the graph, but rather only those mapping onto a meaningful concept within the ontology. The features were carefully selected to ensure that every retained field could be expressed as either a node, an edge, or an interpretable property within the ontology (e.g. artist and track identifiers, title, album, release year, genre tags, or MIDI instrumentation). Derived audio descriptors such as danceability and energy, or popularity scores were excluded from the graph but remained available in the underlying tabular dataset for use in the subsequent modeling and evaluation stages, ensuring that the graph contains only

information that contributes to the relational structure used by the recommender. Out of 40 columns produced by the MSD-Lakh integration, only 22 were kept for the KG.

Similarly, out of the 1,525 musical features extracted per track, 11 were incorporated into the graph as datatype properties on the corresponding Track node (e.g., mrc:meanTempo, mrc:noteDensity). All selected features represented a single, well-defined musical attribute such as mean tempo, note density, amount of staccato, or average number of independent voices. Most of the remaining musical features almost entirely consisted of multi-bin histograms such as pitch, melodic interval, beat, rhythmic value, or instrument-prevalence histograms, whose individual bins did not correspond to musically meaningful concepts, therefore they were excluded from direct representation. Importantly, the selection was not based on a systematic ranking of feature importance, but rather served as an illustrative demonstration of how symbolic music features can be represented within an OWL ontology. The remaining musical features, while musically interpretable, were retained in tabular form and fed in their entirety to the autoencoder, whose compressed output served as the track node feature in the GNN. This decision was made because including these features would clutter the graph without contributing much usable relational information.

The complete list of features included in the knowledge graph, along with their descriptions, origins, and corresponding RDF mappings, is provided in Table 2 in the Appendix.

In the knowledge graph, each row of the merged dataset translates to a connected cluster of nodes representing the entities associated with that recording. Categorical and relational concepts such as genres, instruments, musical keys, modes, and tempo classes were encoded as named individual nodes reused across tracks, enabling the graph to capture shared structure between recordings. Continuous scalar measurements, unique to each track such as tempo, note density, or loudness were, on the other hand, encoded as datatype literals attached to the corresponding track or performance node.

Core node types follow the Music Ontology design: a *Track* carries bibliographic metadata and 11 musical datatype properties (e.g. note density, chromatic motion); an *Artist* links to a *Performance* node that stores tempo and bridges to one of nine categorical *TempoClass* individuals (e.g. Andante, Allegro) and one or more *Instrument* nodes from the MIDI file. Tonal properties—*Key* (12 chromatic values) and *Mode* (Major/Minor)—point to shared named individuals rather than per-track literals, enabling cross-track joins in SPARQL. *Genre* nodes are attached at the artist level and likewise reused across all artists sharing the same tag.

The ontology was further enriched by aligning core classes to the Wikidata upper-concept hierarchy via OWL semantics: `mrc:Genre` ≡ `wd:Q188451`, `mo:Instrument` ≡ `wd:Q34379`, and `mrc:Key` ≡ `wd:Q534932`, with the two mode individuals additionally grounded via `owl:sameAs`. This positions genre, instrument, key, and tempo class within a universally recognised framework, and the shared named individuals create cross-track bridges that let GNN message passing propagate musical context across otherwise disconnected graph regions. Only English labels were retained, as they provided complete coverage across all aligned Wikidata entities.

Release years were bucketed into decade nodes (1960s–2010s), linked into a temporal sequence via `wdt:P155`/`wdt:P156` and grounded to their Wikidata decade entities. This converts a sparse scalar into a shared structural node that hundreds of tracks can co-reference, making the temporal signal exploitable by both SPARQL and GNN message passing.

Finally, the behavioural layer from the Echo Nest Taste Profile was added: each unique user becomes an `mrc:Listener` node (`foaf:Agent` subclass) and each interaction a direct `mrc:listenedTo` edge to the matched track, retaining only rows whose `song_id` already existed in the graph.

A summary of user-related graph components introduced in the behavioral layer is available in Table 3 of the Appendix.

#### 3.1.2. Rich version of the Knowledge Graph

On the top of the presented knowledge graph, a second, richer version of it was additionally created that preserves extra details such as weights, counts, and confidence, preserving all the information from the source dataset. This version matches the Music Ontology's intended modeling much more closely, and allows for more advanced SPARQL inspection and qualitative evaluation, using KG embedding methods that can handle heterogeneous edges, and for frequency-aware recommendation.

In this version, genre associations are reified as blank nodes of type mrc:GenreAssociation, each carrying an mrc:weight value representing the strength of the genre tag, taken either from MSD's published artist_terms_weight or, when missing, from a rank-based fallback. Key and mode are attached to a mo:Performance node rather than directly to the Track, and additionally carry confidence scores (mrc:keyConfidence, mrc:modeConfidence), creating a conceptually cleaner separation and allowing the certainty of the audio analysis to be represented explicitly. Lastly, the listen count is stored as a property of a mrc:ListeningEvent blank node, allowing the model to treat repeated listening as a stronger preference signal than a single play.

Differences between the simple and rich knowledge graph representations in details are represented in Table 4 of the Appendix.

Because the rich variant encodes genre associations and listening events as blank nodes, it was primarily constructed for ontological demonstration and SPARQL inspection in GraphDB rather than as a direct model input. The recommendation model uses the topological structure of the simple KG, extended with edge attributes for relational weights, while all 1,525 musical features are injected via the autoencoder as track node features.

### 3.2. TBox representation

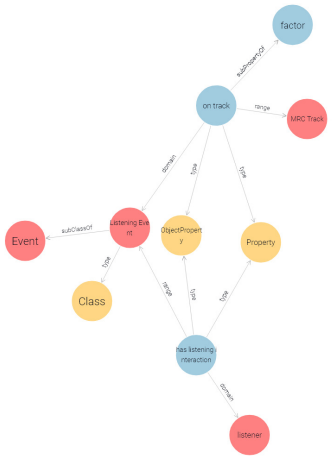


Figure 2. TBox fragment showing the schema-level definition of `mrc:ListeningEvent` and its connecting properties in GraphDB.

Figure 2 shows a fragment of the TBox, illustrating how user listening behaviour is modelled at the schema level. The class `mrc:ListeningEvent` is declared as a subclass of `mo:Event` from the Music Ontology, grounding the vocabulary in an established standard. Two object properties govern its connectivity: `mrc:hasListeningInteraction`, with domain `mrc:Listener` and range `mrc:ListeningEvent`, links a user to their listening events; and `mrc:onTrack`, with domain `mrc:ListeningEvent` and range `mrc:MSDTrack`, associates each event with the track that was played. The latter is further declared a sub-property of `mrc:factor`, reflecting its role as a contributing signal in the recommendation context.

#### 3.2.1. Semantic Inspection

Both KG variants were loaded into GraphDB and validated through SPARQL queries covering genre rankings, confidence-filtered key distributions, ontological hierarchy traversals, the decade temporal chain, and top-listened track lookups. Results are reported in Section 2 and query listings are provided in the Appendix.

### 3.3. From Knowledge Graph to Learnable Tensors

Before any neural model could consume the graph, the RDF triples were converted into a typed tensor representation. A dedicated extraction routine streamed the graph into (i) a consolidated triple table and an edge dictionary, and (ii) a heterogeneous `PyTorch Geometric` graph in which every ontology class becomes a distinct node type (`user`, `track`, `artist`, `genre`, `instrument`, `key`, `mode`, `tempoClass`, `decade`) and every predicate is a typed relation. Track nodes were initialised with the content embeddings produced by the autoencoder described below, and the remaining node types received either knowledge-graph-embedding features or learnable lookup vectors. Because the listening layer of the simple graph encodes each interaction as a direct `user` $\rightarrow$ `track` edge, message passing can reach items in a single hop, which is the configuration used to train the main recommender.

### 3.4. Representation Learning and Recommendation Models

We trained three representation/recommendation components on top of the music knowledge graph, and compared them against three baselines on a shared, stratified user-level interaction split.

#### 3.4.1. Symbolic-feature autoencoder

Because the remaining jSymbolic features per track were too high dimensionally and noisy to use them directly as node features in the GNN, an autoencoder was used to reduce them to a compact, dense representation that still preserves the most important musical structure.

The full 1,526-dimensional jSymbolic vector of each track was standardised and passed through a fully connected autoencoder with encoder $1526 \rightarrow 512 \rightarrow 256 \rightarrow 128$ and a symmetric decoder ($128 \rightarrow 256 \rightarrow 512 \rightarrow 1526$), and with each layer followed by ReLU activation and dropout 0.2. Training used Adam (learning rate $10^{-3}$, batch size 256) to minimise mean-squared reconstruction error, with up to 200 epochs and early stopping once the training loss fails to improve by more than $10^{-4}$ for 15 consecutive epochs. Because the task was pure transductive compression of a fixed catalogue, no held-out ranking split was required. The 128-dimensional bottleneck was exported as a dense content embedding for every track and used both to initialise the track node features of the graph models and as the item representation of the hybrid baseline.

#### 3.4.2. Knowledge-graph embeddings.

Because our KG contains many node types that don't have numeric features from any source, a numeric representation that could be used as input features for the GNN was required. This numeric representation was achieved by training KG embedding models that learn a dense vector for every entity in the graph purely from the graph's relational structure.

Two KG embedding models were trained on triples exported from the enriched KG. RotatE [1], which represents each entity as a complex vector and each relation as a rotation in complex space, and ComplEx [2], which uses a similar complex-valued representation but scores triples via the real part of a three-way interaction between head, relation, and tail entities. Both being well-established complex-valued KGE models that can handle diverse relation types well.

Both models were chosen because our KG is rich in *asymmetric* relations: a user listens to a track but not the reverse; a track belongs to a genre but a genre does not belong to a track; an artist performs a piece but a piece does not perform an artist. Real-valued models such as TransE cannot represent asymmetry, since they place head and tail in the same vector space with a single additive offset that is the same in both directions. Complex-valued representations break this symmetry naturally: in RotatE, a relation $r$ rotates the head entity $h$ to reach the tail $t$ in complex space ($h \circ r \approx t$, element-wise), but the inverse rotation $\bar{r}$ is a distinct transformation, so the model can simultaneously encode $r$ and its asymmetric inverse without conflict. ComplEx achieves asymmetry through the Hermitian dot product $\mathrm{Re}(\langle h, r, \bar{t} \rangle)$, where conjugating $t$ but not $h$ breaks the symmetry of the score.

Despite sharing the complex-space inductive bias, RotatE is expected to be the stronger embedding for this KG due to its ability to model *compositional* relation patterns in addition to symmetry and antisymmetry. Composition arises naturally in our graph: the chain *track* $\xrightarrow{\textit{hasTempoClass}}$ *Allegro* $\xrightarrow{\textit{followedBy}}$ *decade* or genre-hierarchy traversals are all compositional paths. A rotation composed with another rotation is still a rotation, so RotatE can represent such chains geometrically; ComplEx has no analogous closure property and therefore struggles to encode compositional structure. For these reasons RotatE was used as the primary initialiser for the HGT node features, with ComplEx retained for comparison.

Both KG embeddings were trained with entity_dim=128, producing 256-dimensional output vectors per entity, over 100 epochs with Adam optimizer, and with listening events deliberately excluded from the training corpus. The embedding dimension was set to 128 to match the autoencoder bottleneck, ensuring that audio and structural features contribute comparably to the final track representation. Listening interactions, on the other hand, were excluded from the training corpus to avoid leaking the target signal into the node features.

The resulting 256-dimensional embedding for every entity in the graph, were subsequently used to initialise node features for all node types in the graph.

#### 3.4.3. Heterogeneous Graph Transformer (The main model).

The primary recommendation model used in this project was a Heterogeneous Graph Transformer (HGT) [3], selected for its ability to natively operate on graphs containing multiple node and edge types. This was particularly important for our KG since it comprises tracks, artists, users, genres, instruments, keys, modes, tempos, and decades, each representing distinct semantic entities and relationships. Many standard graph neural network architectures aggregate information from neighbouring nodes without explicitly modelling these type differences, potentially obscuring important structural information. In contrast, HGT learns type-specific transformation and attention parameters for each node and edge type combination, enabling it to preserve the heterogeneous structure of the graph throughout the message-passing process and capture more nuanced relationships for recommendation, making it a natural fit for this task.

The HGT stack was configured with two message-passing layers, 4 attention heads, a hidden dimension of 128, and dropout of 0.1. A per-node-type MLP head projects the hidden representations to a 64-dimensional output space, which is L2-normalised to enable cosine similarity scoring, a standard output design for dot-product recommenders. Track nodes were initialised with a 384-dimensional feature vector, the concatenation of the 128-dim autoencoder embedding and the 256-dim KGE embedding, giving the model access to both audio content and relational context. All other node types (artists, genres, instruments, keys, modes, tempos, decades) were initialised from their 256-dim KGE features alone.

The model was trained for 100 epochs using the Adam optimiser ($lr = 10^{-3}$, weight decay $= 10^{-4}$) with a cosine warm-restart learning-rate schedule that periodically reset the learning rate to escape local minima. The checkpoint achieving the best validation Recall@20 was retained for final evaluation.

To address the popularity bias inherent in implicit-feedback data, we adopt an *Intensity-Weighted Listwise Loss with Logit Adjustment*. Given $L_2$-normalised user embedding $\hat{\mathbf{u}}$ and track embeddings $\hat{\mathbf{t}}_i$, the adjusted logit for track $i$ is

$$s_i = \frac{\hat{\mathbf{u}}^\top \hat{\mathbf{t}}_i}{\tau} + \lambda \log p_i, \tag{1}$$

where $\tau$=0.1 sharpens the softmax distribution and $p_i$ is the Laplace-smoothed marginal listen probability of track $i$. Since $\log p_i < 0$, popular tracks receive a downward logit adjustment and less-heard items are relatively boosted, directly discouraging the model from concentrating recommendations on globally popular choices.

Soft training targets per user are constructed from raw play counts as $\tilde{p}_{ui} \propto \log(1 + c_{ui})$, normalised row-wise to a valid probability distribution. The log-transform compresses the count range so that repeat listening contributes a stronger

but diminishing signal. The loss is then the cross-entropy between the adjusted softmax and the soft target distribution, averaged over the user batch:

$$\mathcal{L} = -\frac{1}{|B|} \sum_{u \in B} \sum_{i} \tilde{p}_{ui} \, \log \mathrm{softmax}(\mathbf{s}_u)_i. \qquad (2)$$

This formulation was chosen over binary cross-entropy or BPR because: (i) the listwise objective ranks across the full catalogue simultaneously; (ii) logit adjustment counters the popularity shortcut without heuristic negative sampling; and (iii) soft engagement-weighted targets exploit the ordinal information in play counts. At inference, items are ranked by cosine similarity with training-seen items masked.

#### 3.4.4. Baselines

Three baselines were used to contextualise the performance of the graph-based model, each representing a different level of complexity and a different source of signal.

The first baseline, MostPopular, ranks all unseen items by global play frequency and recommends the most-played tracks to every user identically, serving as a lower bound that any useful recommender should outperform it.

The second baseline, KNN-CF, is a user–user collaborative filter operating on the $\ell 2$-normalised interaction matrix, where similarity between users is measured by cosine distance. For each user, item scores were aggregated from the k most similar neighbours, and unseen items were ranked by this aggregate. The neighbourhood size k was selected via a sweep over candidate values, choosing the k that maximised a composite validation score. This baseline captured behavioural similarity between users without any content or graph information, isolating the contribution of interaction patterns alone. The third baseline, an XGBoost two-stage hybrid, incorporates content features without graph structure. Top-200 candidates per user were retrieved via cosine similarity between a mean-pooled autoencoder user profile and track content embeddings, then re-ranked with a gradient-boosted learning-to-rank model (`rank:ndcg@10`, up to 300 rounds, early stopping at 30) trained on 3,000 sampled users. The 300-dimensional feature vector per (user, track) pair concatenated the 128-dim user-profile and track-AE embeddings with 44 categorical features (one-hot key/mode/tempo class, target-encoded genre and artist, multi-hot instrument indicator). This hybrid fuses content and behaviour without graph structure, directly isolating the GNN's contribution.

#### 3.4.5. Evaluation Protocol

All recommenders were evaluated on a single, shared *user-level stratified* split of the interaction data (training / validation / test), stratified by song-level attributes (genre and release decade) so that the popularity and content distributions were preserved across folds; song features and graph structure were visible to every split, and only the interactions were partitioned, preventing leakage. Knowledge-graph embeddings and the autoencoder are unsupervised and therefore fit on the training data without a held-out ranking split.

We report a standard top-$K$ suite—Recall@$K$, Precision@$K$, NDCG@$K$, Hit-Rate@$K$ and MRR—together with two catalogue *Coverage* and mean *Popularity-Bias*@$K$, and a composite *Overall Score*

$$\begin{aligned}\text{Overall} = 0.6 \cdot \text{NDCG@}K + 0.2 \cdot \text{Coverage} \\ + 0.2 \cdot (1 - \text{PopularityBias@}K), \qquad (3)\end{aligned}$$

which rewards models that are accurate *and* surface diverse, less popular content. Every model was scored with the same recommendation lists sliced at $K \in \{5, 10, 20, 50, 100\}$. Pairwise differences were assessed with the paired Wilcoxon signed-rank test (with Cohen's $d_z$ effect sizes) and an omnibus Friedman test. Because full inference over all 284,864 test users is computationally prohibitive for the XGBoost model, pairwise and omnibus tests were conducted on a shared random subsample of 5,000 users across all models, ensuring a balanced, paired comparison.

## 4. Results

### 4.1. Knowledge Graph Inspection and ABox Visualisation

To verify the structural integrity of the constructed knowledge graph, the rich KG variant was loaded into GraphDB and inspected both visually and through SPARQL queries.

#### 4.1.1. ABox Visualisation

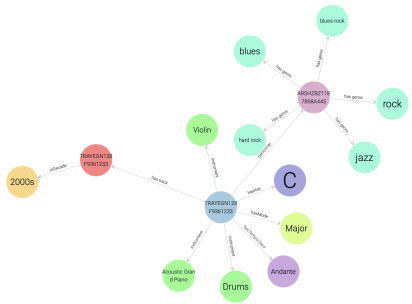


Figure 3. GraphDB visual graph: ABox neighbourhood of a representative track instance (TRAYEGN128F9361233)

Figure 3 shows the ABox neighbourhood of a representative track, *When She Dances* (TRAYEGN128F9361233), as rendered by GraphDB's visual graph explorer. The central `mo:Performance` node fans out via `hasKey`, `hasMode`, and `hasTempoClass` edges (C, Major, Andante), three `mo:instrument` nodes (Violin, Acoustic

Grand Piano, Drums), and a `performer` edge to artist Joe Bonamassa, which in turn links to five genre nodes. A `hasTrack` edge points to the `mrc:MSDTrack` node carrying the `mrc:inDecade` relation to the 2000s decade.

Inspecting the track node's properties (Figures 9–11 in the Appendix) confirmed that all eleven jSymbolic audio features and bibliographic metadata are correctly stored on the `mrc:MSDTrack` instance, verifying that the builder pipeline fully populated both acoustic and descriptive dimensions.

Because, the visual explorer renders only named IRI nodes by default, so the `mrc:GenreAssociation` blank nodes carrying genre weights are not visible in Figure 3, and their presence was confirmed via a dedicated SPARQL query:

```
ASK { ?a mrc:hasGenreAssoc ?assoc .
      ?assoc mrc:weight ?w }
```

which returned `true`. A subsequent `SELECT` query over the same artist retrieved five genre associations with descending weights: blues (1.0), blues-rock (0.6), hard rock (0.51), jazz (0.44), and rock (0.3). These weights reflect the proportional genre affinity of the artist as derived from the MSD tag data, and are stored as `xsd:double` literals on the intermediate blank node.

#### 4.1.2. Knowledge Graph Statistics and SPARQL Inspection

The enriched knowledge graph contains 19,322,668 triples in total, and the node type histogram confirms the expected entity composition: 285,037 listeners, 7,013 tracks, 7,013 performances, 3,668 artists, 1,046 genres, 173 instruments, 14 decades, 12 keys, and 17 tempo classes. Hierarchy traversal queries confirmed that all genre and instrument nodes were correctly linked to their Wikidata super-classes, with genre nodes tracing through music genre and musical concept, and instrument nodes through their specific instrument families up to musical instrument. Similarly, the decade chain query confirmed that all decade nodes from the 1950s to the 2020s were correctly linked via temporal predecessor/successor relations and assigned to their parent century (20th or 21st).

Genre ranking queries revealed that rock is the most frequently occurring genre by artist count (2,117 artists), followed by popular music and pop (1,062 each) and electronic (1,037). When ranked by aggregated weight in the rich KG, the ordering shifts notably: hip-hop subgenres (hardcore hip hop, instrumental hip hop, jazz hip hop, etc.) rise to the top with a total weight of 2,405 across 385 artists, reflecting higher per-artist term weights despite fewer artists than rock, demonstrating how the weighted representation in the rich KG captures genre salience more faithfully than raw artist counts alone.

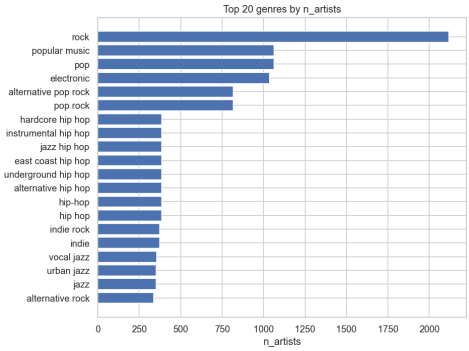


Figure 4. Top 20 genres by n_artists (simple KG)

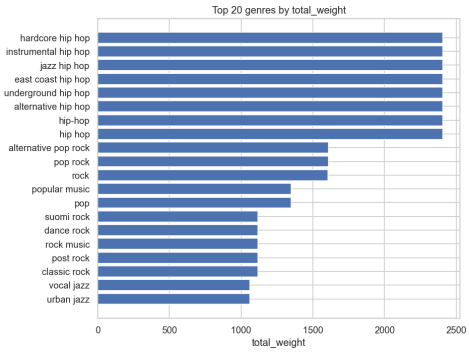


Figure 5. Top 20 genres by total_weight (rich KG)

Key distribution queries on the rich KG, filtered to high-confidence detections ($keyConfidence \geq 0.8$), showed that C major is the most common key (211 tracks), followed by G (196) and D (150), being consistent with the prevalence of these keys in Western popular music.

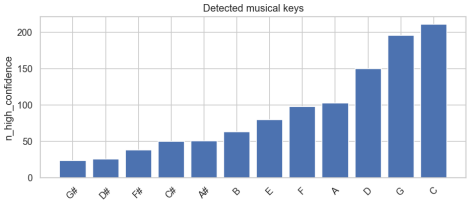


Figure 6. Distribution of High-Confidence Musical Keys

Lastly, through graph exploration we found out that the most listened-to track in the dataset was *Secrets* by OneRepublic, played by 50,058 distinct users with a total of 197,579 plays, followed by *You're The One* by Dwight Yoakam (42,810 listeners, 390,283 total plays, the highest total play count in the dataset).

Overall, the SPARQL inspection confirmed that the knowledge graph was correctly populated and structurally sound, with ontological alignments, temporal chains, and

entity counts all matching the expected design. The genre ranking comparison between the two KG variants illustrated the added expressiveness of the weighted representation, while the key distribution and popularity queries provided initial insights into the musical and behavioural characteristics of the dataset.

### 4.2. Autoencoder Reconstruction Results

The autoencoder trained for the full 200 epochs, as the early stopping criterion - no improvement greater than 10-410≈4 10-4 over 15 consecutive epochs — was never triggered, indicating that the loss continued to decrease, albeit slowly, throughout training. The MSE fell from 0.96 at epoch 0 to 0.62 at epoch 200, with the steepest descent in the first 75 epochs followed by a gradual flattening. The final reconstruction loss of 0.62 confirms that the 128-dimensional bottleneck retains sufficient information to approximately reconstruct the 1,526-dimensional input.

### 4.3. Baseline Model Results

The KNN collaborative filter sweep over the validation set selected $k = 6$ neighbours as optimal, balancing ranking quality against catalogue coverage. At $k = 6$ on the test set, KNN-CF achieved Recall@10 of 0.104, NDCG@10 of 0.106, Hit Rate@10 of 0.170, and MRR of 0.087, with a catalogue coverage of 91.5% and a low popularity bias of 0.133, indicating that the model recommends diverse, non-mainstream items.

The most-popular baseline achieved a comparable Hit Rate@10 of 0.184 and slightly lower NDCG@10 of 0.098. Its catalogue coverage was although only 0.2%, meaning it effectively recommends the same small set of globally popular tracks to every user. Its popularity bias score of 0.628 confirms this. The overall score (a composite metric) was 0.134 for popularity versus 0.420 for KNN-CF, largely driven by the coverage difference.

An XGBoost hybrid reranker was also evaluated at $K = 10$, yielding Recall@10 of 0.038, NDCG@10 of 0.044, and Hit Rate@10 of 0.072, substantially below KNN-CF. Its coverage (66.7%) and low popularity bias (0.118) suggest it diversifies recommendations, but the retrieval bottleneck limits absolute ranking performance.

Together, the baselines establish that a simple collaborative filter already substantially outperforms popularity-based recommendation in ranking quality and catalogue diversity, setting a meaningful bar for the HGT model.

## 5. Conclusion

This work presented a complete pipeline for content- and preference-aware music recommendation grounded in a structured music knowledge graph. We constructed two complementary RDF graph representations of 7,013 tracks with verified ontological alignment to established vocabularies and Wikidata, validated through SPARQL queries in GraphDB. Genre ranking comparisons between the simple and rich KG variants illustrated the expressive value of weighted representations, while decade temporal chains and Wikidata super-class alignment confirmed structural correctness throughout.

A symbolic-feature autoencoder successfully compressed 1,525-dimensional jSymbolic/music21 descriptors to a 128-dimensional bottleneck (final MSE: 0.62), and KG embeddings provided structural node initialisation for all entity types. Baseline evaluation confirms that KNN collaborative filtering substantially outperforms popularity-based recommendation in both ranking quality and catalogue diversity (Overall Score 0.420 vs. 0.134), while the XGBoost hybrid reveals the retrieval bottleneck in content-only approaches without graph structure.

Full training and evaluation of the HGT recommender with the popularity-debiased listwise loss remains ongoing; computational constraints limited the number of experiments that could be completed within the project timeline. Future work will complete the HGT evaluation and compare its performance against the presented baselines, examine the effect of KGE initialisation strategies on recommendation quality, and investigate how Wikidata-grounded graph semantics influence recommendation diversity across musical dimensions.

A structured hyperparameter search—covering the number of HGT attention heads, hidden dimensionality, temperature $\tau$ and debiasing strength $\lambda$ of the listwise loss, and the KGE embedding dimension—would help identify the best-performing configuration and reduce the sensitivity to ad-hoc choices made under time pressure. Complementing this, a rigorous ablation study isolating the individual contributions of KG structure, symbolic audio features, and the popularity-debiased loss would clarify which components drive performance. Extending the baseline suite to include graph-collaborative methods such as LightGCN [10] or KGCN [11], combined with paired statistical significance tests (e.g. Wilcoxon signed-rank on per-user NDCG), would then provide the evidence needed to determine whether the additional complexity of heterogeneous graph modelling is justified for this research question.

## References


[1] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space, 2019. Accepted at ICLR 2019. 1, 5

[2] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction, 2016. Accepted at ICML 2016. 1, 5

[3] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of The Web Conference 2020*, WWW '20, pages 2704–2710, 2020. doi: 10.1145/3366423.3380027. 1, 5

[4] Colin Raffel. *Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching*. PhD thesis, Columbia University, 2016. URL https://colinraffel.com/projects/lmd/. Includes the Lakh MIDI Dataset. 1

[5] Thierry Bertin-Mahieux, Daniel P. W. Ellis, Brian Whitman, and Paul Lamere. The million song dataset. In *Proceedings of the 12th International Society for Music Information Retrieval Conference (ISMIR)*, 2011. URL http://millionsongdataset.com/. 1

[6] The Echo Nest. Taste profile subset of the million song dataset, 2011. URL http://millionsongdataset.com/tasteprofile/. 1

[7] Cory McKay, Julie Cumming, and Ichiro Fujinaga. Jsymbolic 2.2: Extracting features from symbolic music for use in musicological and mir research. In *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, pages 348–354, 2018. 2

[8] Hogue Cuthbert, Ariza and Oberholzer. *music21*. music21, 3rd edition, May 2026. Available at https://music21.org/music21docs/. 2

[9] Yves Raimond, Samer A. Abdallah, Mark B. Sandler, and Frederick Giasson. The music ontology. In *Proceedings of the 8th International Conference on Music Information Retrieval (ISMIR)*, pages 417–422, Vienna, Austria, Sep. 2007. doi: 10.5281/zenodo.1415966.

[10] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. LightGCN: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 639–648, 2020. doi: 10.1145/3397271.3401063. 8

[11] Hongwei Wang, Miao Zhao, Xing Xie, Wenjie Li, and Minyi Guo. Knowledge graph convolutional networks for recommender systems. In *Proceedings of the World Wide Web Conference (WWW)*, pages 3307–3313, 2019. doi: 10.1145/3308558.3313417. 8

# Appendix

## A. Exploratory Overview of MSD Features

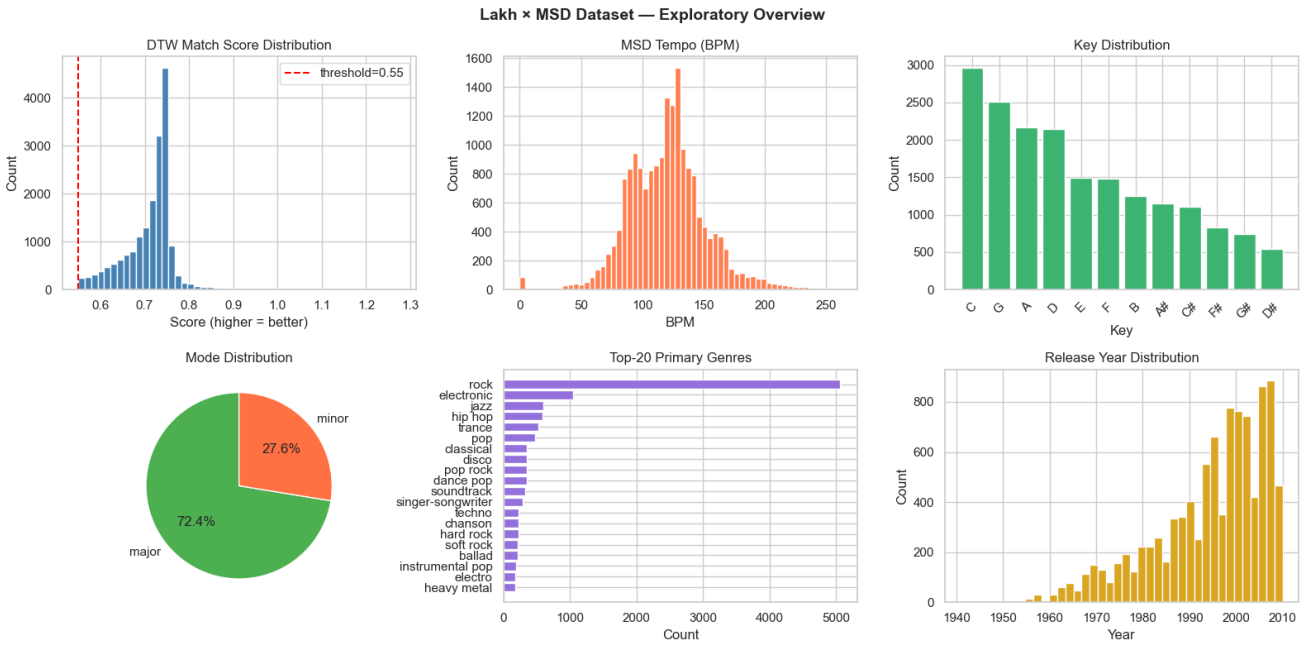


Figure 7. Exploratory Overview of MSD Features.

## B. Exploratory Overview of User Taste Profile Filtering Pipeline and Features

Table 1. User dataset filtering pipeline statistics.

| Step | Interactions | Unique Users | Unique Songs |
|---|---|---|---|
| Raw | 48 373 586 | 1 019 318 | 384 546 |
| – Mismatches | 45 795 100 | 1 019 318 | 378 309 |
| ∩ LMD | 4 243 384 | 848 243 | 7465 |
| – Cold-start | 3 048 959 | 299 156 | 7227 |

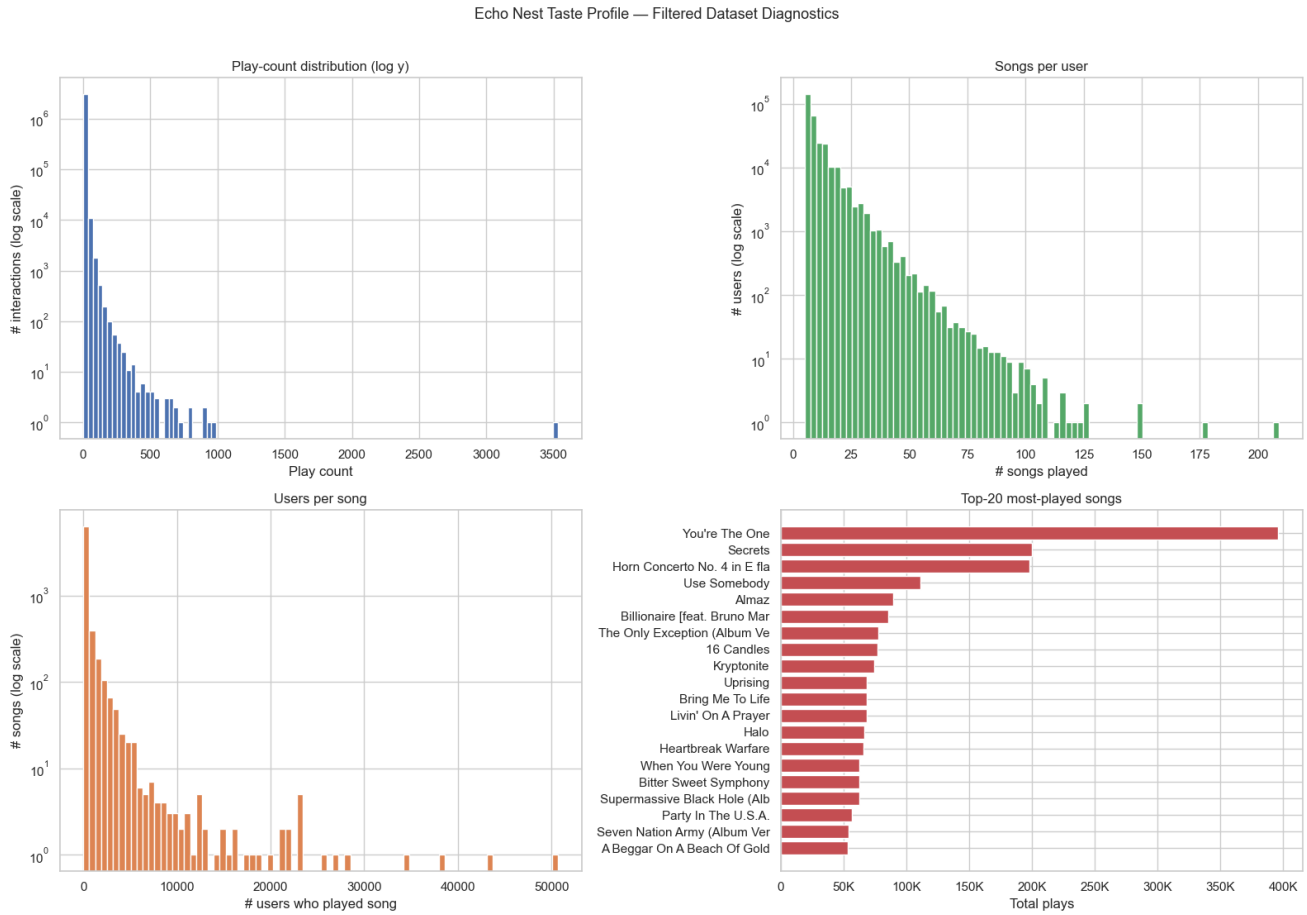


Figure 8. Descriptive statistics of the filtered Echo Nest Taste Profile dataset, including the play-count distribution, songs per user, users per song, and the top-20 most-played songs.

## C. Music-related feature inventory in simple knowledge graph

Table 2. Music related MSD and jSymbolic features mapping to RDF representation

| Feature Name | Description | Origin | RDF Term | Storage Type |
|---|---|---|---|---|
| **track_id** | Unique MSD track identifier used as the primary URI for the track node | MSD | `mo:uuid (literal on track node)` | Literal (xsd:string) |
| **song_id** | Echo Nest song identifier used for cross-referencing with Taste Profile | MSD | `dct:identifier (literal on track node)` | Literal (xsd:string) |
| **artist_id** | MSD canonical artist identifier used as the artist node URI | MSD | `URI slug (artist namespace)` | Named Node (mo:Performer) |
| **artist_mbid** | MusicBrainz cross-reference identifier for the artist | MSD | `mo:musicbrainz_guid` | Literal (xsd:string) |
| **artist_name** | Display name of the artist | MSD | `foaf:name` | Literal (xsd:string) |
| **title** | Title of the track | MSD | `dct:title` | Literal (xsd:string) |
| **album_name** | Name of the release/album the track belongs to | MSD | `dct:isPartOf` | Literal (xsd:string) |
| **year** | Release year of the track | MSD | `dct:date` | Literal (xsd:gYear) |
| **key** | Musical key of the track (C, C#, D ... B) | MSD | `mrc:hasKey` | Named Node (mrc:Key individual) |
| **mode** | Modality of the track (Major or Minor) | MSD | `mrc:hasMode` | Named Node (mrc:Mode individual) |
| **loudness** | Overall loudness of the track in decibels | MSD | `mrc:loudness` | Literal (xsd:double) |

| Feature Name | Description | Origin | RDF Term | Storage Type |
|---|---|---|---|---|
| primary_genre | Highest-weight genre tag of the artist; fallback weight 1.0 when no MSD weight available | MSD | mrc:hasGenre (via artist) | Named Node (mrc:Genre individual) |
| top3_genres | Top 3 genre tags of the artist with rank-based weights | MSD | mrc:hasGenre (via artist) | Named Node (mrc:Genre individual) |
| artist_terms | Full list of MSD genre/style tags with published weights | MSD | mrc:hasGenre (via artist) | Named Node (mrc:Genre individual) |
| midi_instrument_names | List of General MIDI instrument names present in the MIDI file | MIDI (LMD) | mo:instrument (via performance) | Named Node (mo:Instrument individual) |
| tempo (Mean_Tempo / msd_tempo) | Mean BPM of the track; jSymbolic value used when available, MSD scalar as fallback; stored on Performance and used to derive the categorical tempo class | jSymbolic (MSD fallback) | mo:tempo (via performance) | Literal (xsd:double) |
| Tempo class | Categorical tempo marking derived from BPM (e.g. Allegro, Andante) | Derived | mrc:hasTempoClass (via performance) | Named Node (mrc:TempoClass individual) |
| Mean_Tempo | Mean tempo computed by jSymbolic | jSymbolic | mrc:meanTempo | Literal (xsd:double) |
| Initial_Tempo | Tempo at the start of the piece | jSymbolic | mrc:initialTempo | Literal (xsd:double) |
| Tempo_Variability | Variability of tempo across the piece | jSymbolic | mrc:tempoVariability | Literal (xsd:double) |
| Note_Density | Average number of notes per second | jSymbolic | mrc:noteDensity | Literal (xsd:double) |
| Average_Note_Duration | Mean duration of individual notes | jSymbolic | mrc:averageNoteDuration | Literal (xsd:double) |
| Amount_of_Staccato | Proportion of notes that are staccato | jSymbolic | mrc:amountOfStaccato | Literal (xsd:double) |
| Amount_of_Arpeggiation | Degree of arpeggiated melodic motion | jSymbolic | mrc:amountOfArpeggiation | Literal (xsd:double) |
| Chromatic_Motion | Proportion of melodic intervals that are semitones | jSymbolic | mrc:chromaticMotion | Literal (xsd:double) |
| Chord_Duration | Mean duration of chords | jSymbolic | mrc:chordDuration | Literal (xsd:double) |
| Average_Number_of $_{I}ndependent\_Voices$ | Mean number of simultaneous melodic voices | jSymbolic | mrc:averageNumber OfIndependentVoices | Literal (xsd:double) |
| Average_Note_to_Note $_{C}hange\_in\_Dynamics$ | Mean change in dynamics between consecutive notes | jSymbolic | mrc:averageNoteTo NoteChangeIn Dynamics | Literal (xsd:double) |

## D. User-related graph components

Table 3. Taste Profile interaction features mapped to RDF representation.

| Feature Name | Description | Origin | RDF Term | Storage Type |
|---|---|---|---|---|
| user_id | Echo Nest user identifier used as the user node URI | Echo Nest | URI slug (user namespace) | Named Node (mrc:Listener / foaf:Agent) |
| play_count | Number of times a user played a track | Echo Nest | mrc:listenCount | Literal (xsd:integer) |
| user→track interaction | Listening interaction linking a user to a track | Echo Nest | mrc:listenedTo (simple) / mrc:hasListeningInteraction (rich) | Edge (object property) |

Table 4. Differences between the simple and rich knowledge graph representations.

| **Feature** | **Simple KG** | **Rich KG** |
|---|---|---|
| Genre (`artist_terms`, `primary_genre`, `top3_genres`) | Direct edge:<br>`artist`<br>`mrc:hasGenre`<br>`genre` | Blank node:<br>`artist`<br>`mrc:hasGenreAssoc`<br>`[a mrc:GenreAssociation;`<br>`mrc:genre genre;`<br>`mrc:weight w]`<br>Direct edge retained |
| `artist_terms_weight` | Not stored | Stored as `mrc:weight` on the GenreAssociation blank node |
| `key` | Named node on Track (`mrc:hasKey`) | Named node on Performance (`mrc:hasKey`) |
| `mode` | Named node on Track (`mrc:hasMode`) | Named node on Performance (`mrc:hasMode`) |
| `key_confidence` | Not stored | Stored as `mrc:keyConfidence` on Performance |
| `mode_confidence` | Not stored | Stored as `mrc:modeConfidence` on Performance |
| `play_count` | Not stored | Stored as `mrc:listenCount` on the ListeningEvent blank node |
| User→Track interaction | Direct edge:<br>`user`<br>`mrc:listenedTo`<br>`track` | Blank node:<br>`user`<br>`mrc:hasListeningInteraction`<br>`[a mrc:ListeningEvent;`<br>`mrc:onTrack track;`<br>`mrc:listenCount n]` |

**E. Rich Knowledge Graph**

**F. Knowledge Graph Inspection**

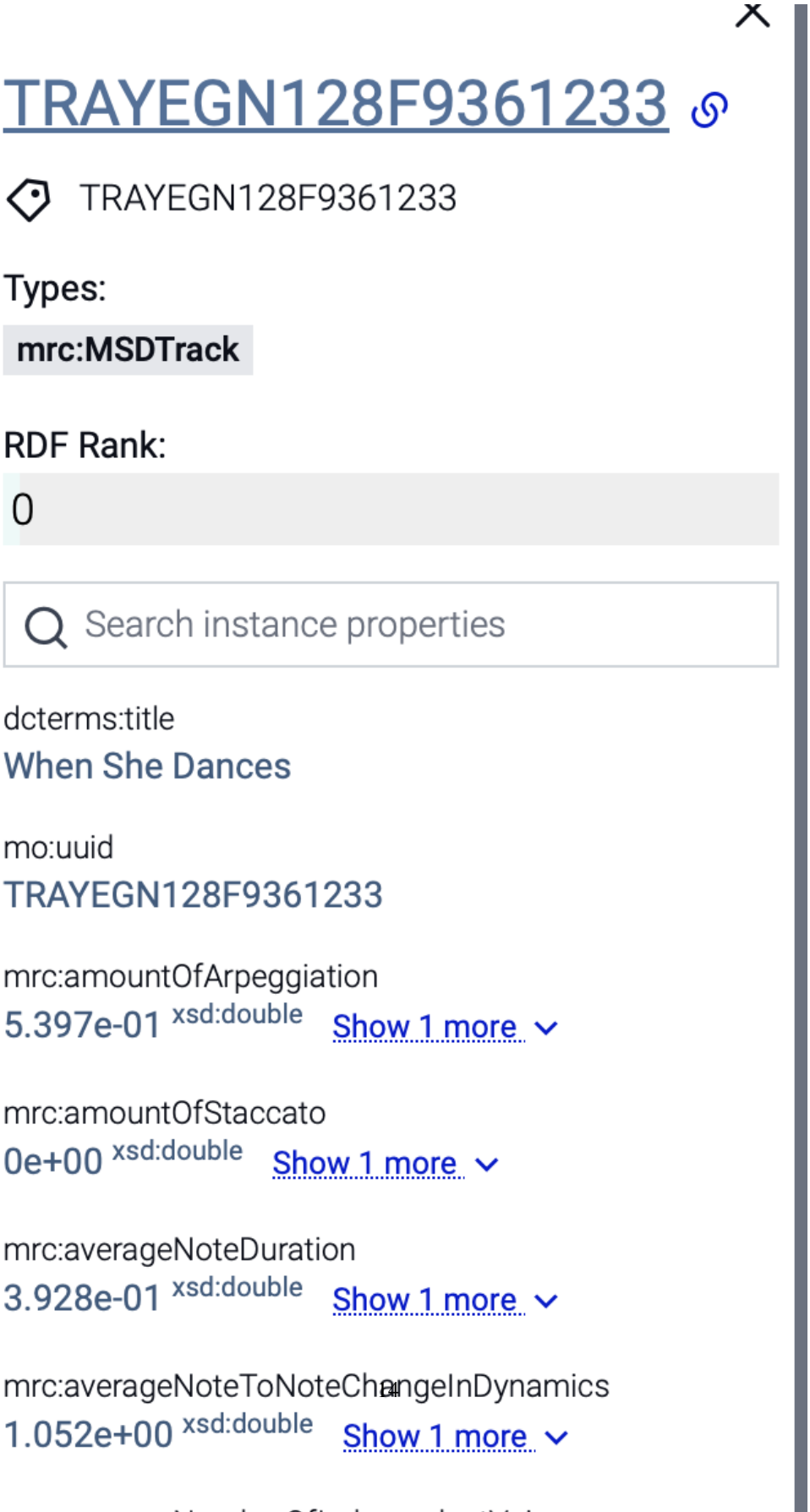


Figure 10. Selected properties of track node TRAYEGN128F9361233 (*When She Dances*) as displayed in GraphDB, showing jSymbolic audio features and metadata stored directly on the `mrc:MSDTrack` instance.

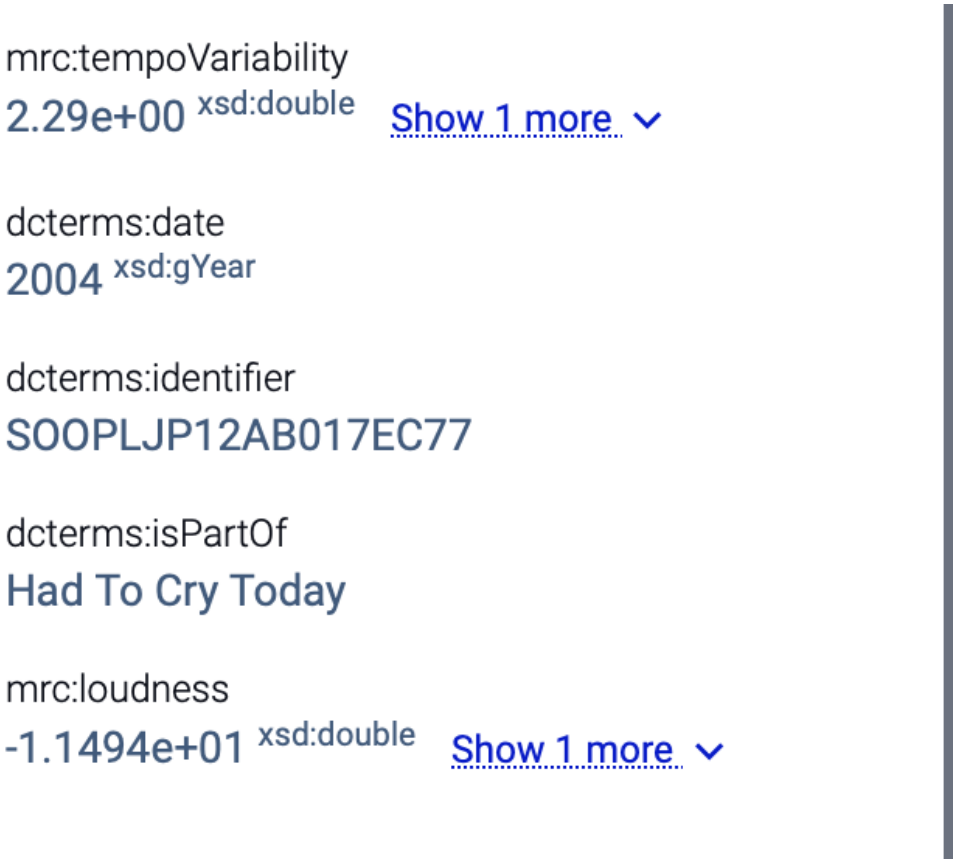


Figure 11. Selected properties of track node TRAYEGN128F9361233 (*When She Dances*) as displayed in GraphDB, showing jSymbolic audio features and metadata stored directly on the `mrc:MSDTrack` instance.

## G. SPARQL Query Examples and Results

All queries were executed against the knowledge graph loaded in GraphDB (or an equivalent in-memory rdflib graph for the simple KG). They share the following common prefix block, which is omitted from individual query listings for brevity:

```
PREFIX mrc:   <http://purl.org/ontology/mrc/>
PREFIX mo:    <http://purl.org/ontology/mo/>
PREFIX rdfs:  <http://www.w3.org/2000/01/rdf-schema#>
PREFIX wdt:   <http://www.wikidata.org/prop/direct/>
PREFIX foaf:  <http://xmlns.com/foaf/0.1/>
PREFIX dct:   <http://purl.org/dc/terms/>
PREFIX owl:   <http://www.w3.org/2002/07/owl#>
```

### Q1 | Genre Ranking, Simple KG

Returns genres ranked by the number of distinct artists tagged with each genre. Direct `mrc:hasGenre` edges are used; no weights are stored in the simple KG.

```
SELECT ?genreLabel (COUNT(DISTINCT ?artist) AS ?n_artists)
WHERE {
    ?artist mrc:hasGenre ?g .
    ?g      rdfs:label ?genreLabel .
    FILTER(LANG(?genreLabel) = "en")
}
GROUP BY ?genreLabel
ORDER BY DESC(?n_artists)
LIMIT 30
```

Table 5. Top 5 genres by artist count (Simple KG, truncated from 30 rows).

| Genre | # Artists |
|---|---|
| rock | 2,117 |
| pop | 1,062 |
| electronic | 1,037 |
| pop rock | 814 |
| hip hop | 381 |

### Q2 | Genre Ranking with Weights, Rich KG

Uses the reified `mrc:GenreAssociation` blank nodes to aggregate per-artist tag weights, revealing salience differences hidden by raw artist counts.

```
SELECT ?genreLabel
       (COUNT(DISTINCT ?artist)       AS ?n_artists)
       (ROUND(SUM(?w) * 1000) / 1000  AS ?total_weight)
       (ROUND(AVG(?w) * 1000) / 1000  AS ?avg_weight)
WHERE {
    ?artist mrc:hasGenreAssoc ?assoc .
    ?assoc  mrc:genre  ?g ;
            mrc:weight ?w .
    ?g      rdfs:label ?genreLabel .
    FILTER(LANG(?genreLabel) = "en")
}
GROUP BY ?genreLabel
ORDER BY DESC(?total_weight)
LIMIT 30
```

Table 6. Top 5 genres by aggregated weight (Rich KG). Note that *electronic* overtakes *pop* compared to Q1, reflecting stronger per-artist tagging.

| Genre | # Artists | Total weight | Avg weight |
|---|---|---|---|
| rock | 2,117 | 1606.8 | 0.759 |
| electronic | 1,037 | 695.3 | 0.671 |
| pop | 1,062 | 674.2 | 0.635 |
| pop rock | 814 | 533.0 | 0.655 |
| hip hop | 381 | 293.1 | 0.769 |

## Q3 | Track Key, Mode and Confidence (Rich KG)

Retrieves the key, mode, tempo, and audio-analysis confidence scores for a specific track via the `mo:Performance` intermediate node. These confidence fields are only available in the rich KG.

```
SELECT ?title ?keyLabel ?modeLabel ?keyConf ?modeConf ?tempo
WHERE {
    BIND(<http://purl.org/ontology/mrc/resource/track/TRBCNFO> AS ?t)
    ?t    dct:title      ?title .
    ?perf mrc:hasTrack   ?t ;
          mrc:hasKey     ?k ;
          mrc:hasMode    ?m .
    OPTIONAL { ?perf mo:tempo           ?tempo    }
    OPTIONAL { ?perf mrc:keyConfidence  ?keyConf  }
    OPTIONAL { ?perf mrc:modeConfidence ?modeConf }
    OPTIONAL { ?k rdfs:label ?keyLabel
               FILTER(LANG(?keyLabel) = "en") }
    OPTIONAL { ?m rdfs:label ?modeLabel
               FILTER(LANG(?modeLabel) = "en") }
}
```

*Example result* (track *Into The Nightlife*):

| Title | Key | Mode | keyConf | modeConf | Tempo |
|---|---|---|---|---|---|
| Into The Nightlife | A | Minor | 0.608 | 0.495 | 122.0 |

## Q4 | Artist Genre Weights via Blank Nodes (Rich KG)

Demonstrates traversal of the reified `mrc:GenreAssociation` blank-node structure, returning genre labels and their MSD tag-strength weights for a specific artist.

```
SELECT DISTINCT ?artistName ?genreLabel ?weight
WHERE {
    BIND(<http://purl.org/ontology/mrc/resource/artist/ARYM37E> AS ?a)
    ?a foaf:name         ?artistName ;
       mrc:hasGenreAssoc ?assoc .
    ?assoc  mrc:genre    ?g ;
            mrc:weight   ?weight .
    ?g      rdfs:label   ?genreLabel .
    FILTER(LANG(?genreLabel) = "en")
}
ORDER BY DESC(?weight)
```

*Example result* (artist *Cyndi Lauper*):

| Artist | Genre | Weight |
|---|---|---|
| Cyndi Lauper | new wave | 1.000 |
| Cyndi Lauper | ballad | 0.673 |
| Cyndi Lauper | rock | 0.600 |
| Cyndi Lauper | soundtrack | 0.525 |
| Cyndi Lauper | pop | 0.300 |

## Q5 | Confidence-Filtered Key Distribution (Rich KG)

Returns the distribution of musical keys restricted to performances where the audio analysis confidence is at least 0.8, filtering out unreliable key estimates.

```
SELECT ?keyLabel (COUNT(*) AS ?n_high_confidence)
WHERE {
    ?perf a mo:Performance ;
          mrc:hasKey        ?k ;
          mrc:keyConfidence ?kc .
    FILTER(?kc >= 0.8)
    ?k rdfs:label ?keyLabel .
    FILTER(LANG(?keyLabel) = "en")
}
GROUP BY ?keyLabel
ORDER BY DESC(?n_high_confidence)
LIMIT 24
```

The query returned 12 key labels. Top results: C (211), G (196), D (150), A (103), F (98), E (80)—consistent with the prevalence of these keys in Western popular music.

## Q6 | Decade Temporal Chain

Verifies the temporal predecessor/successor structure of decade nodes and their alignment to parent century entities via Wikidata properties.

```
SELECT ?prevLabel ?decadeLabel ?nextLabel ?centuryLabel
WHERE {
    ?decade a mrc:Decade ;
            rdfs:label ?decadeLabel .
    FILTER(LANG(?decadeLabel) = "en")
    OPTIONAL {
        ?decade rdfs:subClassOf ?century .
        OPTIONAL { ?century rdfs:label ?centuryLabel
                   FILTER(LANG(?centuryLabel) = "en") }
    }
    OPTIONAL {
        ?decade wdt:P155 ?prev .
        OPTIONAL { ?prev rdfs:label ?prevLabel
                   FILTER(LANG(?prevLabel) = "en") }
    }
    OPTIONAL {
        ?decade wdt:P156 ?next .
        OPTIONAL { ?next rdfs:label ?nextLabel
                   FILTER(LANG(?nextLabel) = "en") }
    }
}
ORDER BY ?decadeLabel
```

*Excerpt of results* (Wikidata IDs shown where English labels were not fetched):

Table 7. Decade temporal chain results (selected rows).

| Prev | Decade | Next | Century |
|---|---|---|---|
| — | 1950s | 1960s | 20th century |
| 1950s | 1960s | 1970s | 20th century |
| 1960s | 1970s | 1980s | 20th century |
| 1970s | 1980s | 1990s | 20th century |
| 1980s | 1990s | 2000s | 20th century |
| 1990s | 2000s | 2010s | 21st century |
| 2000s | 2010s | — | 21st century |